

Project Glossary

Air Quality Index (AQI) Prediction in India

A quick reference guide to every term, function, and model used in the notebook — built to help you discuss the project confidently and answer supervisor questions.

How to use this guide

This document is a reference glossary explaining every term, function, and model that appears in the notebook (final_modified.ipynb) for the AQI prediction project, organized in the same order as the project's workflow — from loading the data to saving the final model. Use it for quick review before your defense, and to answer any technical term with confidence.

01 Libraries & Tools

NumPy

Numerical computing

Python's core library for array operations and numerical math; used here for calculations such as taking the square root when computing RMSE.

Pandas

Tabular data handling

The main library used to load city_day.csv into a DataFrame and perform filtering, groupby aggregation, merging, and date handling.

Matplotlib / Seaborn

Plotting & visualization

Used to draw distribution histograms, boxplots, correlation heatmaps, and time-series plots of average AQI.

Scikit-learn (sklearn)

Core ML library

Provides all the modeling tools used in the project: train/test splitting, linear and tree-based models, evaluation metrics, and grid search tuning.

XGBoost / LightGBM / CatBoost

Gradient boosting libraries

Specialized high-performance gradient boosting frameworks. XGBoost outperformed every other model and was selected as the final model.

KaggleHub

Dataset download

A package used to download the latest version of the India Air Quality dataset directly from Kaggle inside the notebook.

```
kagglehub.dataset_download("rohanrao/air-quality-data-in-india")
```

joblib

Model persistence

Used to serialize the trained model into a .pkl file so it can be reloaded and reused later without retraining.

```
joblib.dump(best_xgb, "xgboost_model.pkl")
```

tqdm

Progress bar

Displays a progress bar while looping through and training multiple candidate models.

02 Dataset & Target Variable

city_day.csv*Main data file*

Contains daily pollutant readings and AQI values for several Indian cities over an extended period — the project's primary data source.

AQI (Air Quality Index)*Target variable*

A single number summarizing air pollution level based on multiple pollutant concentrations; the value every model tries to predict.

AQI_Bucket*AQI category label*

A text column classifying AQI into categories such as Good, Satisfactory, Moderate, Poor, Very Poor, and Severe.

City / Date*Location & time*

Two core columns identifying where and when each reading was taken, later used for temporal and spatial trend analysis.

Pollutants*PM2.5, PM10, NO, NO2, NOx, NH3, CO, SO2, O3, Benzene, Toluene, Xylene*

Columns representing the concentration of each chemical or particulate pollutant. Used as independent features to predict AQI; PM2.5 and PM10 turned out to be the most influential.

03 Feature Engineering**Feature Engineering***General term*

The process of creating new columns (features) from raw data to improve model learning — here done by extracting temporal information from the Date column.

Year / Month / Day*Extracted date parts*

Columns extracted from the Date column using `pd.to_datetime` so the model can capture yearly and monthly patterns in pollution.

```
df['Date'].dt.year / .dt.month / .dt.day
```

DayOfWeek*Weekday index*

A number representing the day of the week (0=Monday ... 6=Sunday), helping detect differences between weekdays and weekends.

Quarter*Calendar quarter*

Identifies which quarter (1 to 4) the date falls in, used to analyze broader seasonal differences than a single month.

04 Exploratory Data Analysis (EDA)

Missing Values

Null / NaN entries

Empty cells in the dataset, checked with `isnull().sum()` and later handled through median imputation before training.

Histogram

Distribution plot

A chart showing the distribution of AQI or a pollutant, used to see where values cluster and whether outliers exist.

Boxplot

Spread / outlier plot

Shows the median, quartiles, and outliers of AQI across the top 10 most polluted cities, used to compare spread between cities.

Correlation Heatmap

Correlation matrix plot

A color matrix showing the correlation coefficient between every pair of numeric variables, used to spot which pollutants relate most strongly to AQI.

```
df[numeric_cols].corr()
```

Time Series Analysis

Trend over time

Studying how average AQI evolves over time (daily, monthly, yearly) to uncover overall trends and recurring seasonal patterns.

Resampling

Time aggregation

Aggregating daily readings into monthly averages using `resample('M')` to smooth out daily fluctuations and reveal the overall trend.

05 Data Preprocessing

SimpleImputer

Missing-value imputation

An sklearn tool that fills missing pollutant values using the median strategy, ensuring rows aren't lost due to incomplete data.

```
SimpleImputer(strategy='median')
```

train_test_split

Train/test splitting

Splits the data into 80% training and 20% testing, ensuring the model is evaluated on data it hasn't seen during training.

```
train_test_split(X, y, test_size=0.2, random_state=42)
```

StandardScaler

Feature scaling

Rescales numeric variables to zero mean and unit variance; commonly used with scale-sensitive models like KNN and SVM.

LabelEncoder

Categorical encoding

Converts text values (e.g., city names or AQI categories) into integers that a model can process.

06 Baseline Regression Models

Linear Regression

Simplest baseline

Assumes a straight-line relationship between pollutants and AQI; used as a baseline for comparison against other models.

Lasso / Ridge

Regularized regression (L1 / L2)

Variants of linear regression that add a penalty term to reduce complexity and prevent overfitting; Lasso can zero-out some coefficients, Ridge only shrinks them.

KNN (K-Nearest Neighbors)

Distance-based model

Predicts AQI based on the average of the 5 nearest, most similar data points, using the Minkowski distance metric.

```
KNeighborsRegressor(n_neighbors=5, metric='minkowski', p=1)
```

SVM / SVR

Support Vector Regression

Tries to find the best prediction function within a defined error margin; used here with a linear kernel.

Decision Tree

Rule-based splits

Splits data into branches based on sequential conditions on pollutant values to predict AQI; max_depth=10 was set to avoid excessive complexity.

Random Forest

Ensemble of trees

A large collection of decision trees trained on different random samples of data; the final prediction averages all trees, reducing variance and improving accuracy.

Gradient Boosting

Sequential ensemble

Builds trees sequentially, where each new tree corrects the errors of previous ones, gradually improving performance.

XGBoost

The project's winning model

An optimized, fast implementation of gradient boosting; achieved the highest accuracy of all models tested and was chosen as the final model after tuning.

LightGBM / CatBoost

Additional boosting models

Alternative gradient boosting libraries known for speed (LightGBM) and strong handling of categorical data (CatBoost); used as base models inside the Stacking ensemble.

07 Regression Evaluation Metrics

MAE (Mean Absolute Error)

Average absolute error

The average absolute difference between actual and predicted AQI values. Lower is better, and it's easy to interpret since it's in the same unit as AQI.

MSE (Mean Squared Error)

Average squared error

The average of squared differences between actual and predicted values; penalizes large errors more heavily than MAE.

RMSE (Root Mean Squared Error)

Square root of MSE

The square root of MSE, bringing the error back to the original AQI unit for easier interpretation.

R² Score

Coefficient of determination

Shows the percentage of variance in AQI that the model can explain using the independent variables; closer to 1 means a better model.

08 Ensemble Models: Voting & Stacking

Voting Regressor

Simple/weighted averaging

Combines predictions from several models (e.g., Linear Regression, Random Forest, Decision Tree) using simple or weighted averaging for a more stable final prediction.

Stacking Regressor

Two-level ensemble

An advanced ensemble that trains several base models (Random Forest, Gradient Boosting, CatBoost, LightGBM), then feeds their predictions into a meta-model (XGBoost here) that learns how to best combine them.

Meta-model

Combiner model

The model responsible for learning how to combine the outputs of the base models inside Stacking to reach the best final prediction.

09 Hyperparameter Tuning

GridSearchCV

Grid search with cross-validation

Automatically tries every possible combination of parameters defined in `param_grid`, evaluating each combination via cross-validation to pick the best XGBoost configuration.

```
GridSearchCV(estimator=xgb_model, param_grid=param_grid, cv=5, scoring='r2')
```

param_grid**Parameter grid**

A dictionary specifying candidate values for each parameter to tune, such as `n_estimators`, `max_depth`, `learning_rate`, `subsample`, and `colsample_bytree`.

Cross-Validation (cv)**K-fold validation**

Splits training data into 5 folds and trains/tests the model multiple times on different folds for a more reliable evaluation, less dependent on chance.

best_params_**Best parameter set**

The attribute returning the best parameter combination found after `GridSearchCV` finishes.

best_estimator_**Best trained model**

The final model already trained with the best discovered parameters, ready to be used directly for prediction.

10 Feature Importance & Model Persistence

Feature Importance**Contribution score**

A value measuring how much each variable (e.g., PM2.5 or PM10) contributes to the final model's decisions; results show PM2.5 and PM10 as the most influential in predicting AQI.

`best_xgb.feature_importances_`

Top Features Selection**Feature subset selection**

Keeps only the top 5 most important variables (instead of all of them) to retrain the model in a simpler, faster way without a major loss in accuracy.

Model Persistence (joblib)**Saving the final model**

Storing the final trained model in `xgboost_model.pkl` so it can later be used in an application or prediction interface without retraining from scratch.

11 Anticipated Q&A; for Project Defense

1. Why did you choose XGBoost as the final model? Because it achieved the highest R^2 and lowest MAE/RMSE compared to all baseline and ensemble models tested.
2. Why use median instead of mean for missing-value imputation? Because the median is less sensitive to the outliers commonly found in pollutant readings.
3. What's the difference between Voting and Stacking? Voting combines predictions directly by averaging, while Stacking uses an additional meta-model that learns how to combine them intelligently.
4. Why were PM2.5 and PM10 the most important features? Because they are the fine particulates most physically and statistically tied to the official AQI calculation.
5. What is GridSearchCV useful for? Automatically trying multiple values for each parameter and selecting the best one, instead of manual, random trial and error.